

Metodos numericos

Jaime Nazar Anchorena

Contents

Capitulo 1: Aproximacion y error	2
Calculo numerico	2
Ejemplo	2
Error	2
Formatos de salida Octave	2
Numeros en punto flotantes	2
Numeros especiales	3
Formula para pasar a decimal	3
Unidad 2: Soluciones aproximadas de ecuaciones no lineales	3
Metodo babilonico	3
Metodo de incrementos	3
Metodo de biseccion	4
Error	4
Limitaciones del metodo	5
Metodo Newton Raphson	5
Teorema de Newton-Raphson	5
Teorema	5
Error	5

Capitulo 1: Aproximacion y error

Calculo numerico

Desarrollo de **metodos constructivos** para la solucion numerica de problemas matematicos. Problemas clasicos como calcular la integral, resolver un sistema lineal, etc. Estos problemas suelen ser dificil de encontrar una solucion mediante el calculo, entonces se utilizan aproximaciones(se conoce como **solucion numerica**).

Ejemplo

Sea $f : [a, b] \rightarrow \mathbb{R}$ una funcion continua en $[a, b]$ derivable en (a, b) tal que $f(a) f(b) < 0$ y signo (f') constante en (a, b) . Entonces f tiene un **unico cero** en (a, b) .

Esto tiene solucion, pero no me dice donde encontrarla. Para ello, debemos encontrar un procedimiento en un numero finito de pasos, pero casi nunca se encuentra la solucion.

Error

Obviamente, aparecera un error, puede ser absoluto o relativo.

- Error absoluto = | valor aproximado - valor exacto |
- Error relativo = error absoluto / valor exacto (si valor exacto $\neq 0$)

Como el valor exacto no sera conocido, solo se podran establecer cotas para el error. Hay dos factores que inducen error:

- De aproximacion o truncado(algoritmo-dependiente)
- De redondeo(maquina-dependiente, no puedo guardar un numero periodico en una computadoras)

Formatos de salida Octave

- **format**: 5 cifras significativas
- **format long**: 15 cifras significativas
- **format long e**: Formato exponencial
- **format bit**: Formato de punto flotante de doble precision(en binario)

Numeros en punto flotantes

La implementacion solo puede representar un subconjunto finito de numeros racionales y no estan distribuidos en la recta real. De manera que hay un maximo numero posible de representar.

Mas especificamente, dentro de la computadora, se utiliza la norma IEEE 754. Hay muy pocos numeros que se pueden representar sin error.

La distancia entre dos numeros consecutivos crece al aumentar el numero considerado. Donde:

$$x_i = (1 + m) \cdot 2^E$$

Y el siguiente:

$$x_{i+1} = (1 + m + \text{eps}) \cdot 2^E$$

Donde m es la mantisa y E el exponente. Además, eps es el epsilon de la computadora, el incremento mas chico posible.

Numeros especiales

- **Cero:** Se representa con todos 0
- **eps:** Incremento mas chico posible. 2^{-52}
- **realmin:** Numero mas chico posible. 2^{-1022}
- **realmax:** Numero mas grande posible.
- **Inf:** Cualquier numero mayor a realmax. Sale cuando, por ejemplo, se divide por 0

Formula para pasar a decimal

$$(-1)^{\text{bit signo}} \cdot 2^{E-1023} \cdot (1 + m)$$

Donde E es el exponente y m la mantisa.

El error relativo de esta implementacion es $\frac{1}{4} \cdot \text{eps}$

Unidad 2: Soluciones aproximadas de ecuaciones no lineales

Vamos a ver metodos para hallar raices de ecuaciones de tipo:

$$f(x) = 0$$

donde f es una funcion.

Salvo casos muy simples es practicamente imposible hallar una solucion exacta a estas ecuaciones, en general la teoria matematica nos puede decir si estas soluciones existen y mas o menos donde se encuentran, pero no nos dice como hallarlas.

Se buscan metodos que permitan hallar soluciones aproximadas y tener una cota para el error. El ejemplo es encontrar la raiz cuadrada de un numero.

Metodo babilonico

La idea es la siguiente:

Si $x^s = 2$ entonces $x = \frac{2}{x}$, Si x_0 es un numero, entonces el promedio entre x_0 y $\frac{2}{x_0}$ deberia estar mas cerca de la solucion que x_0 .

Entonces, a partir de una sucesion, se puede llegar a una solucion.

Metodo de incrementos

Metodo de fuerza bruta, se va calculando los valores de la funcion en incrementos muy chicos y por teorema de Bolzano se puede acotar un intervalo donde se encuentra la raiz.

Metodo de biseccion

Primero, se debe determinar un intervalo donde se encuentra la raiz(aplicando Teorema de Bolzano). El algoritmo es el siguiente:

Se define una sucesio s_n que converge a la solucion.

1. Tomamos $s_0 = \frac{b_0+s_0}{2}$. El promedio entre a_0 y b_0 . Es la primer aproximacoin de la raiz. El error es menor que $\frac{L}{2}$.
2. Miramos el signo de $f(s_0)$. Si es positivo tiene que haber una raiz entre a_0 y s_0 . En ese caso llamamos $a_1 = a_0$ y $b_1 = s_0$. Si es negativo debe haber una raiz entre s_0 y b_0 . En ese caso llamamos $a_1 = s_0$ y $b_1 = b_0$. Ahora repetimos el paso 1. Es decir, tomamos $s_1 = \frac{b_1+a_1}{2}$ y ahora el error es menor que $\frac{L}{4}$.
3. Repetir hasta que el error sea menos que el pedido.

Se llama biseccion pues en cada paso partimos a la mitad.

Como sabemos que va a funcionar? Por la convergencia, en cada paso puedo asegurar que:

1. $a_n \leq a_{n+1} < b_0$
2. $a_0 < b_{n+1} \leq b_n$
3. $a_n < s_n < b_n$
4. $|b_n - a_n| \rightarrow 0$
5. Entonces existe un unico c tal que $a_n \rightarrow c$, $b_n \rightarrow c$ y por lo tanto $s_n \rightarrow c$. Como f es continua $f(c) = 0$.

Error

Sea $L = b_0 - a_0$, en el n paso el error es menor que $\frac{L}{2^{n+1}}$. Luego, se puede deducir cuantos pasos necesitamos para llegar al error pedido, si r es el verdadero valor buscado, entonces:

$$|r - s_n| \leq \frac{L}{2^{n+1}}$$

Si queremos asegurar que el error cometido es menor que una constante $\epsilon > 0$ hay que tomar n tal que:

$$\frac{L}{2^{n+1}} < \epsilon$$

Es equivalente a pedir que:

$$\frac{L}{\epsilon} < 2^{n+1}$$

Tomando logaritmo base 2 y haciendo cambio de base se obtiene la formula final.

Si f es una funcion continua, $a < b$, $f(a) \cdot f(b) < 0$, y llamamos $L = b - a$. Si queremos hallar la solucion de $f(x) = 0$ con un error menor que un numero $\epsilon > 0$ por el metodo de biseccion deberemos calcular s_n con:

$$n = \lceil \ln_2\left(\frac{L}{\epsilon} - 1\right) \rceil$$

donde para un numero real positivo x el numero $[x]$ denota un numero netural n tal que $n - 1 \leq x < n$.

Limitaciones del metodo

Cuando el cero esta en un minimo o maximos local de f el metodo tiene problemas.

Metodo Newton Raphson

Tiene mas cosas que pueden ir mal, pero es mucho mas eficiente. La idea es usar la recta tangente cada vez mas cerca al cero para encontrar la raiz. La tangente sale de los polinomios de Taylor.

Teorema de Newton-Raphson

Sea f una funcion de clase C^2 en $[a, b]$ si existe $a < p < b$ tal que $f(p) = 0$ y $f'(p) \neq 0$ entonces existe $\delta > 0$ tal que si $|x_0 - p| < \delta$, entonces la sucesion:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

converge a p .

Tiene algunas desventajas. Hay que hayar primero una raiz aproximada y en general hay que ver que $f'(x) \neq 0$ en el intervalo en consideracion.

Si no se cumplen las hipotesis puede fallar extremadamente fuerte.

Teorema

Esto es lo que vamos a usar.

Sea f una funcion de clase C^2 en $[a, b]$ si existe $a < x_0 < b$ tal que $f(x_0) = 0$, y los signos de f' y f'' son constantes entonces la sucesion generada por el metodo de Newton Raphson converge a cero unico de f en (a, b) en forma monotona si se elige como valor inicial s_0 al extremo de $[a, b]$ donde $\text{sign}(f'')$ y $\text{sign}(f)$ son iguales en ese extremo.

Se puede demostrar(ver apuntes para ver demo)

Error

Supongamos que el metodo Newton-Raphson genera una sucesion x_n que converge a una raiz p de f . Sea $E_n = |x_n - p|$ entonces:

1. Si p es una raiz simple entonces

$$E_{n+1} \approx \left| \frac{f''(p)}{2 \cdot f'(p)} \right| \cdot E_n^2$$